Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра системного проектування

ЗВІТ ДО ЛАБОРАТОРНОЇ РОБОТИ №10
«Production Python: Docker, CI/CD та захист проєкту»
з навчальної дисципліни «Професійний Python»

Варіант №8 — «Система бронювання готелю (Hotel Reservation System)»

Виконав: студент 3 курсу
групи ФеП-32
Івасів Данило

Перевірив: доцент
Куньо Іван Михайлович
Дата: 20.10.2026

Львів — 2026

1. Мета роботи

Ознайомитися з підходами до підготовки Python-проєкту до production-oriented використання. Навчитися оформлювати Python-проєкт як інстальований пакет за стандартом PEP 517 / PEP 621; створювати бінарні wheel (.whl) та вихідні sdist (.tar.gz) дистрибутиви; використовувати pyproject.toml для централізованої декларації залежностей та інструментів; працювати зі змінними середовища (environment variables) та дотримуватися принципів безпеки (не зберігати secrets у коді, використовувати .env та .env.example).

Опанувати контейнеризацію: створювати оптимізовані багатоетапні Dockerfile (multi-stage build), забезпечувати роботу від безпечного непривілейованого користувача (non-root user), будувати Docker image, запускати Docker container, передавати конфігурацію у контейнер, налаштовувати Docker volumes для персистентності БД та інструкцію самодіагностики HEALTHCHECK. Створити автоматизований CI/CD pipeline у GitHub Actions для автоматичного запуску лінтингу (Ruff), перевірки форматування, статичної типізації (Mypy), модульного тестування (Pytest) з контролем порогу тестового покриття (coverage threshold >= 80%), збирання дистрибутивів та контейнерного образу; підготувати проєкт до фінального захисту.

2. Індивідуальний варіант і постановка задачі

Згідно з індивідуальним Варіантом №8 продовжується розвиток проєкту «Система бронювання готелю (Hotel Reservation System)».
Основні дані системи:
• Номер готелю (Room): номер (room_number), категорія/тип (room_type: single, double, suite, deluxe), ціна за добу (price), статус (status: available, occupied, maintenance);
• Гість (Guest): ім'я (first_name), прізвище (last_name), унікальний email, телефон (phone);
• Бронювання (Reservation): номер кімнати (room_number), ім'я гостя (guest_name), ідентифікатор гостя (guest_id), дата заїзду (check_in), дата виїзду (check_out), загальна сума (total_price), статус (status: confirmed, cancelled).

Обов'язкова предметна функціональність:
1. Пошук доступних номерів на вказаний інтервал дат без колізій (get_available_rooms);
2. Створення бронювання з алгоритмічною перевіркою конфліктів дат та запобіганням подвійного бронювання (book_room / double-booking prevention);
3. Скасування активного бронювання (cancel_reservation);
4. Пошук найдешевшого діючого номера серед доступних або всього фонду (find_cheapest_room);
5. Розрахунок потенційної та фактичної виручки готелю (calculate_potential_revenue);
6. Перевірка перетину інтервалів дат (has_date_conflict).

Вимоги до Production-стану проєкту (Лабораторна робота №10):
• Оформити як production package/module: hotel-booking / hotel_booking;
• Зберегти та задокументувати REST API, створене у ЛР №8 (/rooms, /rooms/available, /guests,

/reservations) та обов'язково додати системний ендпоінт GET /health;
• Використати persistence layer з БД SQLite (SQLAlchemy 2.0 ORM, Repository pattern) та оптимізації аналітики з ЛР №9 (NumPy векторизація, LRU кешування з інвалідацією);
• Конфігурацію БД та застосунку передавати через environment variables (DATABASE_URL, ENVIRONMENT, LOG_LEVEL, API_HOST, API_PORT);
• Забезпечити безпеку: жодних реальних паролів, токенів чи API ключів у вихідному коді, README або Git. Створити .env.example та додати .env у .gitignore;
• Створити pyproject.toml metadata, зібрати wheel та sdist дистрибутиви, верифікувати встановлення wheel у чистому ізольованому середовищі;
• Створити Dockerfile (multi-stage build, non-root user, HEALTHCHECK) та .dockerignore, налаштувати збереження даних через volume;
• Налаштувати GitHub Actions CI workflow (ruff check, ruff format, mypy, pytest >= 80% coverage, package build, docker build);
• Оновити README.md, описати сценарій захисту та підготувати проєкт до демонстрації.

3. Архітектура системи та структура фінального проєкту (src-layout)

Фінальний проєкт організовано відповідно до сучасного стандарту src-layout, де весь виконуваний код ізольовано в окремий пакет src/hotel_booking, що запобігає випадковому імпорту локальних модулів замість встановленого пакета. Архітектура системи базується на багатошаровому підході (Layered Architecture):
• API Layer (FastAPI): Маршрутизація HTTP-запитів, валідація схем через Pydantic v2, обробка помилок та ендпоінт моніторингу /health;
• Service Layer (HotelService): Інкапсуляція бізнес-правил, захист від подвійного бронювання та аналітичні підрахунки;
• Repository Layer (RoomRepository, GuestRepository, ReservationRepository): Абстракція доступу до бази даних SQLite;
• Database Layer (SQLAlchemy 2.0): Декларативні ORM-моделі, керування сесіями та ініціалізація схеми;
• Configuration & Logging: Централізовані параметри через Pydantic Settings та структуроване production-логування;
• Analytics Layer: Векторизовані статистичні підрахунки на базі NumPy та кешування результатів із контролем інвалідації.

```
Lab_10/
├── .github/
│   └── workflows/
│       └── ci.yml             # GitHub Actions CI/CD пайплайн (matrix 3.11/3.12)
│
├── src/
│   └── hotel_booking/         # Головний пакет предметної області
│       ├── __init__.py        # Експорт моделей, сервісів та константи __version__ =
"1.0.0"
│       ├── api.py             # FastAPI REST API, lifespan, обробники винятків, GET
/health
│       ├── config.py          # Pydantic Settings, змінні оточення, налаштування
логування
│       ├── database.py        # SQLAlchemy Base, engine, SessionLocal, init_db
│       ├── models.py          # ORM сутності: Room, Guest, Reservation
│       ├── schemas.py         # Pydantic v2 схеми валідації запитів та серіалізації
```

```
відповідей
│       ├── exceptions.py      # Доменні винятки: ConflictException,
EntityNotFoundException
│       ├── repositories.py    # Репозиторії доступу до даних: Room, Guest, Reservation
│       ├── services.py        # Бізнес-логіка: доступність, double-booking, розрахунок
виручки
│       ├── analytics.py       # NumPy векторизація та LRU кешування з інвалідацією
│       └── main.py            # CLI інтерфейс (hotel-booking) та автоматичне сідування
БД
│
├── tests/                     # Повний тестовий набір (36 тестів, 97% покриття)
│   ├── __init__.py
│   ├── conftest.py            # Ізольована in-memory SQLite фікстура та TestClient
│   ├── test_config.py         # Тестування конфігурації та env-змінних
│   ├── test_health.py         # Тестування /health та кореневого ендпоінта
│   ├── test_models.py         # Тестування ORM моделей, розрахунку ночей
│   ├── test_repositories.py   # Тестування CRUD та пошуку колізій у БД
│   ├── test_services.py       # Тестування бізнес-правил, перевірки перетинів дат
│   ├── test_api.py            # Інтеграційні тести REST API endpoints
│   ├── test_analytics.py      # Тестування NumPy обчислень та кешування
│   └── test_main.py           # Тестування CLI прапорців та сідування
│
├── benchmarks/
│   ├── __init__.py
│   └── benchmark.py           # Бенчмарк Python vs NumPy векторизації
│
├── migrations/
│   ├── __init__.py
│   └── README.md              # Документація структури міграцій БД
│
├── Dockerfile                 # Multi-stage, non-root user (appuser), HEALTHCHECK
├── docker-compose.yml         # Специфікація Docker Compose з томом даних hotel_data
├── .dockerignore              # Файли, що виключаються з образу Docker
├── .gitignore                 # Виключення тимчасових файлів, .env, *.db, dist/, build/
├── .env.example               # Безпечний шаблон змінних середовища
├── pyproject.toml             # Декларація пакета, метаданих та інструментів
├── README.md                  # Головна документація проєкту та сценарій захисту
├── LICENSE                    # Ліцензія MIT
└── alembic.ini                # Конфігурація для міграцій Alembic
```

4. Упаковка проєкту (Packaging), pyproject.toml та дистрибутиви

Згідно зі стандартами PEP 517, PEP 518 та PEP 621, конфігурація збирання проєкту зосереджена у файлі pyproject.toml. Як бекенд збирання використано setuptools.build_meta із декларацією базових і додаткових залежностей (dev-залежностей). Пакет зареєстровано під назвою hotel-booking версії 1.0.0 із точкою входу CLI: hotel-booking = hotel_booking.main:main.

```
[build-system]
requires = ["setuptools>=70", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "hotel-booking"
version = "1.0.0"
description = "Production-oriented Hotel Reservation System"
readme = "README.md"
requires-python = ">=3.11"
license = "MIT"
keywords = ["hotel", "booking", "fastapi", "docker", "ci-cd", "production"]
dependencies = [
    "fastapi>=0.110.0",
    "uvicorn>=0.30.0",
    "sqlalchemy>=2.0.0",
```

```
    "pydantic[email]>=2.0.0",
    "pydantic-settings>=2.0.0",
    "httpx>=0.27.0",
    "numpy>=2.0.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=8.0.0",
    "pytest-asyncio>=0.23.0",
    "pytest-cov>=5.0.0",
    "ruff>=0.5.0",
    "mypy>=1.10.0",
    "build>=1.2.0",
]

[project.scripts]
hotel-booking = "hotel_booking.main:main"

[tool.setuptools.packages.find]
where = ["src"]

[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = ["E", "F", "W", "I", "B"]
ignore = ["B008", "E501"]

[tool.mypy]
python_version = "3.11"
ignore_missing_imports = true
warn_unused_configs = true
disallow_untyped_defs = true

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
python_files = ["test_*.py"]
addopts = "-v --cov=hotel_booking --cov-report=term-missing"
```

Результат збирання дистрибутивів (uv build / python -m build):

У результаті виконання команди збирання в директорії dist/ сформовано два стандартні дистрибутивні артефакти:
1. Бінарний пакет Wheel: dist/hotel_booking-1.0.0-py3-none-any.whl (готовий до миттєвого встановлення);
2. Вихідний архів Source Distribution: dist/hotel_booking-1.0.0.tar.gz (містить вихідний код, тести та маніфест).

```
dist/
├── hotel_booking-1.0.0-py3-none-any.whl   (24 810 bytes)
└── hotel_booking-1.0.0.tar.gz            (21 540 bytes)
```

Верифікація встановлення wheel у чистому ізольованому середовищі:

Для підтвердження повної відтворюваності пакет було встановлено у щойно створене чисте віртуальне оточення test_env:
Команда: uv venv test_env && uv pip install --python test_env dist/hotel_booking-1.0.0-py3-none-any.whl
Результат перевірки точки входу: ./test_env/Scripts/hotel-booking --version -> Hotel Booking

System v1.0.0. Пакет є повністю самодостатнім і встановлюється разом із усіма транзитивними залежностями.

5. Конфігурація через змінні оточення та безпечна робота із Secrets

Відповідно до методології 12-Factor App, конфігурація системи повністю відокремлена від вихідного коду. Для типобезпечного парсингу та валідації параметрів середовища застосовано модуль pydantic_settings.BaseSettings. Керування налаштуваннями оформлено у вигляді кешованого синглтона через декоратор @lru_cache.

```
# src/hotel_booking/config.py
from functools import lru_cache
import logging
import os
from pydantic import Field
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    app_name: str = Field(default="Hotel Booking System", validation_alias="APP_NAME")
    environment: str = Field(default="development", validation_alias="ENVIRONMENT")
    database_url: str = Field(default="sqlite:///hotel.db",
validation_alias="DATABASE_URL")
    log_level: str = Field(default="INFO", validation_alias="LOG_LEVEL")
    api_host: str = Field(default="0.0.0.0", validation_alias="API_HOST")
    api_port: int = Field(default=8000, validation_alias="API_PORT")
    secret_key: str = Field(
        default="insecure-dev-secret-key-change-in-production",
        validation_alias="SECRET_KEY",
    )

    model_config = SettingsConfigDict(
        env_file=".env",
        env_file_encoding="utf-8",
        extra="ignore",
    )

@lru_cache
def get_settings() -> Settings:
    if "APP_ENV" in os.environ and "ENVIRONMENT" not in os.environ:
        os.environ["ENVIRONMENT"] = os.environ["APP_ENV"]
    return Settings()
```

Файл-зразок змінних оточення (.env.example):

```
APP_NAME=Hotel Booking System
ENVIRONMENT=development
DATABASE_URL=sqlite:///hotel.db
LOG_LEVEL=INFO
API_HOST=0.0.0.0
API_PORT=8000
SECRET_KEY=change-me-to-a-secure-random-secret-key-in-production
```

Аудит безпеки (Security Audit Checklist):

• У вихідному коді відсутні зашиті конфіденційні паролі або токени;

• Файл .env занесено у .gitignore, що унеможливлює його випадкове потрапляння у відкритий Git-репозиторій;

• У репозиторії опубліковано виключно публічний безпечний шаблон .env.example з описом необхідних параметрів;

• У Dockerfile не копіюються файли середовища розробника завдяки правилам у .dockerignore.

6. Контейнеризація застосунку (Docker & Docker Compose)

Для контейнеризації системи створено сучасний оптимізований Dockerfile, що реалізує завдання підвищеної складності:
1. Multi-stage build: Складається з етапу builder (де компілюється wheel-пакет) та етапу runner (де встановлюється виключно сформований артефакт на базі python:3.12-slim);
2. Non-root user: Застосунок виконується від імені непривілейованого користувача appuser із системною безпекою;
3. Docker HEALTHCHECK: Контейнер самостійно перевіряє стан працездатності кожні 30 секунд через внутрішній виклик urllib до ендпоінта /health;
4. Persistent Volume: Директорія /app/data виділена під змонтований том для безпечного збереження файлу БД SQLite.

```
# Dockerfile
FROM python:3.12-slim AS builder
WORKDIR /build
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
COPY pyproject.toml README.md ./
COPY src ./src
RUN python -m pip install --no-cache-dir --upgrade pip build && \
    python -m build --wheel

FROM python:3.12-slim AS runner
WORKDIR /app
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \
    ENVIRONMENT=production LOG_LEVEL=INFO \
    DATABASE_URL=sqlite:////app/data/hotel.db API_HOST=0.0.0.0 API_PORT=8000

RUN useradd --create-home --shell /bin/bash appuser && \
    mkdir -p /app/data && chown -R appuser:appuser /app

COPY --from=builder /build/dist/*.whl /tmp/
RUN python -m pip install --no-cache-dir --upgrade pip && \
    python -m pip install --no-cache-dir /tmp/*.whl && \
    rm -rf /tmp/*.whl

USER appuser
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "hotel_booking.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

Файл виключень .dockerignore:

```
.git
.github
.venv
.env
__pycache__
*.pyc
.pytest_cache
.mypy_cache
.ruff_cache
tests
benchmarks
dist
build
*.db
```

Специфікація Docker Compose (docker-compose.yml):

```
version: "3.8"
services:
  hotel-booking:
    build:
      context: .
      dockerfile: Dockerfile
    image: hotel-booking:1.0.0
    container_name: hotel_booking_app
    restart: unless-stopped
    ports:
      - "8000:8000"
    environment:
      - APP_NAME=Hotel Booking System
      - ENVIRONMENT=production
      - LOG_LEVEL=INFO
      - DATABASE_URL=sqlite:////app/data/hotel.db
    volumes:
      - hotel_data:/app/data

volumes:
  hotel_data:
    name: hotel_booking_data
```

Команди складання, запуску та верифікації контейнера:

1. Складання образу: docker build -t hotel-booking:1.0.0 .
2. Запуск контейнера: docker run --rm -d -p 8000:8000 -v hotel_data:/app/data hotel-booking:1.0.0
3. Перевірка відповіді ендпоінта /health: curl http://localhost:8000/health
Відповідь: {"status": "ok", "environment": "production", "version": "1.0.0", "app_name": "Hotel Booking System"}
4. Статус Healthcheck у системі Docker: Healthy (контейнер успішно пройшов автоматичну самодіагностику).

7. Реалізовані REST API ендпоінти (Варіант 8)

Усі ендпоінти забезпечують сувору типізацію та повертають семантичні статус-коди HTTP: 200 OK, 201 Created, 204 No Content, 400 Bad Request (помилка дат), 404 Not Found (сутність відсутня), 409 Conflict (double-booking). Документація генерується автоматично у форматах Swagger UI (/docs), ReDoc (/redoc) та OpenAPI JSON (/openapi.json).

Метод	URL Маршрут	Призначення ендпоінта	Статус-код
GET	/health	Перевірка життєздатності сервісу, середовища та версії	200 OK
GET	/	Кореневий інформаційний ендпоінт із посиланнями на документацію	200 OK
GET	/rooms	Пагінований список номерів із фільтрами за типом, ціною та статусом	200 OK
POST	/rooms	Створення нового номера в готелі (захист від дублювання номера)	201 Created / 409
GET	/rooms/{id}	Детальна інформація про номер за його унікальним ID	200 OK / 404

PATCH	/rooms/{id}	Часткове оновлення вартості, категорії чи статусу кімнати	200 OK / 404
DELETE	/rooms/{id}	Видалення номерного запису з бази даних	204 No Content / 404
GET	/rooms/available	Пошук вільних номерів за діапазоном check_in та check_out	200 OK / 400
GET	/rooms/cheapest	Пошук найдешевшого діючого номера (з фільтром категорії)	200 OK / 404
GET	/rooms/revenue	Підрахунок виручки та векторизована NumPy аналітика цін	200 OK
POST	/guests	Реєстрація гостя з валідацією пошти (перевірка унікальності)	201 Created / 409
GET	/guests	Список зареєстрованих гостей із пошуком за іменем та email	200 OK
GET	/guests/{id}	Отримання профілю гостя за його ідентифікатором	200 OK / 404
PATCH	/guests/{id}	Оновлення контактних даних гостя (телефону, імені)	200 OK / 404 / 409
DELETE	/guests/{id}	Видалення профілю гостя	204 No Content / 404
POST	/reservations	Створення бронювання з алгоритмічним захистом від колізій	201 Created / 409
GET	/reservations	Список усіх бронювань із фільтрацією за кімнатою та статусом	200 OK
GET	/reservations/{id}	Отримання повної інформації про бронювання	200 OK / 404
DELETE	/reservations/{id}	Скасування активного бронювання (переведення в cancelled)	204 No Content / 404

8. Автоматизоване тестування та аналіз тестового покриття

Для проєкту розроблено вичерпний набір із 36 автоматизованих тестів на базі фреймворку pytest. Для тестування використовується ізольована база даних SQLite у пам'яті (:memory:) зі StaticPool, що забезпечує повну незалежність кожного тестового кейсу. Тести покривають конфігурацію, ORM-моделі, репозиторії, бізнес-правила double-booking prevention, інтеграційні HTTP-запити через TestClient та CLI.

```
============================= test session starts ==============================
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.7.0
rootdir: C:\Projects\Labs\Pro_python_Labs\Lab_10
configfile: pyproject.toml
testpaths: tests
plugins: anyio-4.15.1, asyncio-1.4.0, cov-7.1.0
```

```
collected 36 items

tests/test_analytics.py::test_calculate_descriptive_statistics_empty PASSED [  2%]
tests/test_analytics.py::test_calculate_descriptive_statistics_values PASSED [  5%]
tests/test_analytics.py::test_calculate_vectorized_revenue PASSED        [  8%]
tests/test_analytics.py::test_cached_hotel_analytics_lifecycle PASSED    [ 11%]
tests/test_api.py::test_rooms_api_crud_flow PASSED                       [ 13%]
tests/test_api.py::test_available_and_cheapest_rooms_api PASSED          [ 16%]
tests/test_api.py::test_revenue_statistics_api PASSED                    [ 19%]
tests/test_api.py::test_guests_api_crud_flow PASSED                      [ 22%]
tests/test_api.py::test_reservations_api_double_booking_prevention PASSED [ 25%]
tests/test_config.py::test_default_settings PASSED                       [ 27%]
tests/test_config.py::test_environment_override PASSED                   [ 30%]
tests/test_config.py::test_app_env_alias PASSED                          [ 33%]
tests/test_config.py::test_setup_logging PASSED                          [ 36%]
tests/test_health.py::test_health_endpoint PASSED                        [ 38%]
tests/test_health.py::test_root_endpoint PASSED                          [ 41%]
tests/test_main.py::test_seed_database PASSED                            [ 44%]
tests/test_main.py::test_cli_version_flag PASSED                         [ 47%]
tests/test_main.py::test_cli_seed_only_flag PASSED                       [ 50%]
tests/test_models.py::test_room_model_attributes_and_repr PASSED         [ 52%]
tests/test_models.py::test_guest_model_attributes_and_full_name PASSED   [ 55%]
tests/test_models.py::test_reservation_model_nights_calculation PASSED   [ 58%]
tests/test_models.py::test_reservation_minimum_one_night PASSED          [ 61%]
tests/test_repositories.py::test_room_repository_crud PASSED             [ 63%]
tests/test_repositories.py::test_room_repository_find_cheapest PASSED    [ 66%]
tests/test_repositories.py::test_guest_repository_crud PASSED            [ 69%]
tests/test_repositories.py::test_reservation_repository_crud_and_conflicts PASSED [ 72%]
tests/test_services.py::test_has_date_conflict_logic PASSED              [ 75%]
tests/test_services.py::test_create_room_and_validations PASSED          [ 77%]
tests/test_services.py::test_get_room_and_not_found PASSED               [ 80%]
tests/test_services.py::test_update_and_delete_room PASSED               [ 83%]
tests/test_services.py::test_get_available_rooms_filtering PASSED        [ 86%]
tests/test_services.py::test_find_cheapest_room PASSED                   [ 88%]
tests/test_services.py::test_guest_crud_and_validations PASSED           [ 91%]
tests/test_services.py::test_book_room_double_booking_prevention PASSED  [ 94%]
tests/test_services.py::test_cancel_and_delete_reservation PASSED        [ 97%]
tests/test_services.py::test_calculate_potential_revenue PASSED          [100%]

============================== tests coverage ==============================
Name                                 Stmts   Miss  Cover   Missing
------------------------------------------------------------------
src\hotel_booking\__init__.py            6      0   100%
src\hotel_booking\analytics.py          26      0   100%
src\hotel_booking\api.py               128      0   100%
src\hotel_booking\config.py             24      0   100%
src\hotel_booking\database.py           23      4    83%   38-42
src\hotel_booking\exceptions.py         14      0   100%
src\hotel_booking\main.py               48      7    85%   38, 47, 52-53, 114-115, 125
src\hotel_booking\models.py             46      0   100%
src\hotel_booking\repositories.py      132      2    98%   59, 61
src\hotel_booking\schemas.py            56      1    98%   102
src\hotel_booking\services.py          147      6    96%   153, 206-209, 213
------------------------------------------------------------------
TOTAL                                  650     20    97%
======================== 36 passed in 0.67s ========================
```

Висновки тестування: Сумарне тестове покриття становить 97% (630 із 650 інструкцій), що повністю задовольняє вимогу Section 88–89 щодо coverage >= 80%.

9. Контроль якості коду (Linting, Formatting, Type Checking)

Для забезпечення найвищої якості коду та дотримання стандартів PEP 8 у проєкті налаштовано сучасний набір статичних аналізаторів: Ruff (linter + formatter) та Mypy (строга

типізація).

Результати локальної та CI-перевірки:
1. Ruff Linter (uv run ruff check .): All checks passed! 0 попереджень та 0 помилок;
2. Ruff Formatter (uv run ruff format --check .): 26 files already formatted;
3. Mypy Type Checking (uv run mypy src): Success: no issues found in 11 source files.

10. Неперервна інтеграція та доставка (CI/CD у GitHub Actions)

У директорії .github/workflows/ci.yml налаштовано повністю автоматизований пайплайн перевірки, який спрацьовує при кожному push та pull_request у гілки main і develop. Пайплайн містить матрицю версій Python (3.11 та 3.12), виконує повний аудит коду, перевіряє поріг покриття тестами, збирає пакети wheel/sdist, публікує їх як GitHub Artifacts та будує Docker образ.

```
name: CI

on:
  push:
    branches:
      - main
      - develop
  pull_request:
    branches:
      - main
      - develop

jobs:
  quality-and-test:
    name: Lint, Test, Build & Containerize
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        python-version: ["3.11", "3.12"]

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Set up Python ${{ matrix.python-version }}
        uses: actions/setup-python@v5
        with:
          python-version: ${{ matrix.python-version }}
          cache: "pip"

      - name: Upgrade pip and install build tools
        run: |
          python -m pip install --upgrade pip
          python -m pip install build

      - name: Install dependencies in editable mode
        run: |
          pip install -e ".[dev]"

      - name: Ruff Lint Check
        run: |
          ruff check .

      - name: Ruff Format Check
```

```
      run: |
        ruff format --check .

    - name: Type Checking with mypy
      run: |
        mypy src

    - name: Run Test Suite with Coverage (fail under 80%)
      run: |
        pytest -v --cov=hotel_booking --cov-report=term-missing --cov-fail-under=80

    - name: Build Package (wheel and sdist)
      run: |
        python -m build

    - name: Verify Distribution Artifacts
      run: |
        ls -la dist

    - name: Upload Build Artifacts
      if: matrix.python-version == '3.12'
      uses: actions/upload-artifact@v4
      with:
        name: python-dist-packages
        path: dist/*
        retention-days: 7

    - name: Build Docker Image
      if: matrix.python-version == '3.12'
      run: |
        docker build -t hotel-booking:ci .
```

Робота із GitHub Secrets: Для захищеного деплою або публікації секрети (наприклад, PYPI_TOKEN, DOCKER_TOKEN) зберігаються у GitHub Repository Settings -> Secrets and variables -> Actions та підставляються у workflow через вираз env: TOKEN: ${{ secrets.DOCKER_TOKEN }}. Жодних токенів у тексті workflow не зашито.

11. Історія Git-комітів та сценарій демонстрації на захисті

Відповідно до вимог Section 81–82, репозиторій містить чітку послідовність атомарних логічних комітів та фінальний релізний тег v1.0.0.

```
aecfb6b (HEAD -> main, tag: v1.0.0) Add project documentation
884a9f8 Add CI pipeline
ac90e82 Add Docker support
47048ba Add Python packaging
5a9aca9 Add production configuration
c083488 Initial project structure and domain business logic
```

Покроковий сценарій захисту проєкту (Section 80):

• Крок 1. Структура: Продемонструвати модульний src-layout (src/hotel_booking, tests, Dockerfile, pyproject.toml);
• Крок 2. Безпека: Пояснити роботу src/hotel_booking/config.py, .env.example та наявність .env у .gitignore;
• Крок 3. Метадані: Відкрити pyproject.toml, показати залежності, точку входу hotel-booking та стандарти PEP 517/621;
• Крок 4. Тестування: Запустити uv run pytest -v — продемонструвати проходження 36 тестів із 97% покриттям;
• Крок 5. Якість коду: Запустити uv run ruff check . та uv run mypy src — показати нуль

помилок;
• Крок 6. Пакетування: Виконати uv build — продемонструвати наявність .whl та .tar.gz у каталозі dist/;
• Крок 7. Контейнеризація: Виконати docker build -t hotel-booking:1.0.0 . (multi-stage, non-root user);
• Крок 8. Запуск: Запустити docker run із томом даних та відкрити браузер;
• Крок 9. Ендпоінти: Продемонструвати GET /health (status: ok) та інтерактивний Swagger UI на /docs;
• Крок 10. CI/CD: Показати зелені чек-марки GitHub Actions Pipeline (lint, type check, tests, package build, docker build).

12. Контрольний список готовності (Production Checklist)

Критерій перевірки	**Вимога індустріального стандарту**	**Статус у проєкті**
Локальний запуск	Застосунок стартує через CLI / uvicorn без помилок	Виконано [x]
Автоматичні тести	100% тестів проходять успішно (36 із 36)	Виконано [x]
Тестове покриття	Покриття коду >= 80% (факт: 97%)	Виконано [x]
Статичний лінтинг	Ruff check не знаходить жодних стильових порушень	Виконано [x]
Форматування коду	Ruff format відповідає стандарту 88 символів	Виконано [x]
Статична типізація	Муру суворо перевіряє анотації типів без помилок	Виконано [x]
Складання Wheel	Бінарний пакет .whl створюється успішно	Виконано [x]
Складання Sdist	Вихідний архів .tar.gz створюється успішно	Виконано [x]
Встановлення Wheel	Перевірено встановлення wheel у чистий venv	Виконано [x]
Безпека Secrets	Жодних реальних паролів у Git; наявний .env.example	Виконано [x]
Ігнорування .env	Файл .env надійно занесено до .gitignore	Виконано [x]
Multi-stage Docker	Оптимізований збирач та мінімальний образ runner	Виконано [x]
Безпечний non-root	Процес у контейнері запущено від appuser	Виконано [x]
Docker Healthcheck	Інструкція HEALTHCHECK опитує ендпоінт /health	Виконано [x]
Persistent Volume	SQLite зберігається у /app/data поза контейнером	Виконано [x]
Ендпоінт /health	Повертає status: ok та активне оточення	Виконано [x]
REST API Swagger	Інтерактивна документація на /docs та /redoc	Виконано [x]
Green CI Pipeline	GitHub Actions успішно проходить	Виконано [x]

	усі перевірки	
Логічні коміти	Історія Git структурована та містить тег v1.0.0	Виконано [x]

13. Відповіді на контрольні запитання (Section 93)

1. Що таке Python package?

Каталог файлів, що містить файл __init__.py (або namespace package) та модулі Python, що дозволяє структурувати код за просторами імен та імпортувати його через крапкову нотацію.

2. Що таке distribution package?

Запакований версіонований архів (wheel або sdist), що містить вихідний або скомпільований код, метадані та інструкції встановлення, готовий для публікації на PyPI або передачі клієнтам.

3. Що таке wheel?

Стандартний бінарний формат дистрибуції Python-пакетів (PEP 427). Він містить готові файли для безпосереднього копіювання у site-packages без виконання setup.py під час встановлення.

4. Яке розширення має wheel?

Файли формату wheel завжди мають розширення .whl (наприклад, hotel_booking-1.0.0-py3-none-any.whl).

5. Що таке sdist?

Source distribution (вихідний дистрибутив), зазвичай у форматі .tar.gz, що містить повний вихідний сирцевий код, конфігурацію pyproject.toml та метадані для подальшого збирання на машині користувача.

6. Для чого використовується pyproject.toml?

Єдиний декларативний конфігураційний файл проєкту (PEP 517, 518, 621), що визначає бекенд збирання, метадані пакету, залежності, точки входу CLI та налаштування лінтерів, типізації й тестів.

7. Що містить [build-system]?

Вказує вимоги до інструментів збирання (параметр requires, наприклад ['setuptools>=70', 'wheel']) та бекенд компіляції/пакування (параметр build-backend, наприклад 'setuptools.build_meta').

8. Що містить [project]?

Основні стандартизовані метадані: назву (name), версію (version), опис (description), ліцензію (license), версію Python (requires-python), перелік залежностей (dependencies) та авторів.

9. Як створити wheel?

Виконавши стандартну команду збирання: python -m build --wheel або через пакетний менеджер: uv build --wheel.

10. Як створити sdist?

Виконавши команду створення вихідного дистрибутива: python -m build --sdist або uv build --sdist.

11. Для чого використовується python -m build?

Офіційний стандартний фронтенд для збирання пакетів (PEP 517), що автоматично створює ізольоване віртуальне середовище, встановлює білд-залежності та генерує wheel і sdist.

12. Що таке semantic versioning?

Система версіонування програмного забезпечення у трикомпонентному форматі MAJOR.MINOR.PATCH, де кожна цифра має чітко регламентоване змістовне значення.

13. Що означає MAJOR?

Перша цифра версії; збільшується при внесенні несумісних змін у публічний API (breaking changes), що вимагають адаптації коду споживачів.

14. Що означає MINOR?

Друга цифра версії; збільшується при додаванні нової функціональності зі збереженням повної зворотної сумісності з попередніми випусками.

15. Що означає PATCH?

Третя цифра версії; збільшується при зворотно-сумісному виправленні виявлених дефектів та багів без додавання нового функціоналу.

16. Що таке environment variable?

Динамічно пойменоване строкове значення в контексті операційної системи процесу, що дозволяє передавати параметри конфігурації без потреби модифікації коду програми.

17. Навіщо configuration виносити з коду?

Для забезпечення принципу 12-Factor App, ізоляції конфіденційних даних (секретів), можливості швидкої зміни середовищ (dev, test, prod) та відсутності необхідності повторного збирання коду.

18. Що таке secret?

Конфіденційні цифрові дані (паролі, приватні ключі, API-токени, сертифікати), компрометація яких надає несанкціонований доступ до системи та несе прямі загрози безпеці.

19. Наведіть приклади secrets.

Пароль до облікового запису БД, приватний ключ SSH/TLS, API-ключ платіжного шлюзу (Stripe), JWT secret key підпису сесій, access token до реєстру образів Docker або PyPI.

20. Чому secrets не можна commit у Git?

Історія комітів Git зберігається перманентно і реплікується на всі клоновані копії. Секрет стає доступним усім користувачам репозиторію та публічним ботам-сканерам, що призводить до негайного зламу.

21. Для чого використовується .env?

Текстовий файл для локальної розробки, що містить пари КЛЮЧ=ЗНАЧЕННЯ та автоматично підтягується застосунком у змінні середовища під час старту.

22. Чому .env має бути у .gitignore?

Оскільки файл .env створюється розробником локально і може містити реальні паролі чи персональні налаштування, його публікація у системі контролю версій суворо заборонена.

23. Для чого потрібний .env.example?

Файл-шаблон у Git, який демонструє перелік усіх необхідних змінних середовища для запуску проєкту із фіктивними безпечними прикладами значень.

24. Що таке Docker?

Платформа контейнеризації на рівні операційної системи, що дозволяє ізолювати застосунок разом з його системними бібліотеками, залежностями та середовищем у компактний контейнер.

25. Що таке Dockerfile?

Текстовий файл-інструкція, який покроково описує команди складання Docker-образу (базовий образ, копіювання файлів, встановлення залежностей, права доступу та команду старту).

26. Що таке Docker image?

Незмінний статичний read-only шаблон, побудований за Dockerfile, який містить файлову систему, середовище виконання, код програми та метадані для запуску.

27. Що таке Docker container?

Запущений у ізольованому просторі (namespace, cgroups) екземпляр Docker image, що має власний мережевий інтерфейс, процеси та тонкий записуваний шар.

28. Чим image відрізняється від container?

Image — це пасивний збережений шаблон (як скомпільований бінарний файл або клас в ООП), а Container — це активний виконуваний екземпляр цього образу (як запущений процес або об'єкт класу).

29. Для чого використовується docker build?

Для автоматизованого збирання Docker image на основі вказаного каталогу з контекстом та інструкцій файлу Dockerfile.

30. Для чого використовується docker run?

Для створення та запуску нового контейнера з вказаного Docker image із можливістю конфігурації портів, змінних середовища та томів даних.

31. Що робить EXPOSE?

Декларує мережевий порт, який процес усередині контейнера прослуховує за замовчуванням (слугує документацією та підказкою для публікації через -p чи зв'язку між контейнерами).

32. Що робить CMD?

Вказує команду та аргументи за замовчуванням, які автоматично виконуються при старті контейнера (може бути перевизначена аргументами команди docker run).

33. Для чого потрібний WORKDIR?

Встановлює робочий каталог усередині файлової системи контейнера для всіх наступних інструкцій COPY, RUN, CMD та ENTRYPOINT.

34. Що робить COPY?

Копіює файли або директорії з файлової системи хоста (з контексту збирання) у вказаний каталог файлової системи створюваного Docker image.

35. Для чого використовується .dockerignore?

Файл шаблонів виключень, який запобігає надсиланню непотрібних файлів (.git, .venv, .env, кеші) у контекст збирання Docker, зменшуючи розмір образу та час збирання.

36. Що таке Docker volume?

Механізм персистентного збереження даних поза життєвим циклом контейнера, який монтує область диска хоста всередину контейнера та зберігає стан при його видаленні/перезапуску.

37. Для чого потрібний health endpoint?

Спеціальний легкий HTTP-маршрут (GET /health), за допомогою якого оркестратори (Docker, Kubernetes) або балансувальники періодично перевіряють працездатність процесу.

38. Що таке CI?

Continuous Integration (неперервна інтеграція) — практика регулярного автоматичного злиття змін коду розробників у спільну гілку із негайним запуском лінтерів, компіляції та автотестів.

39. Що таке CD?

Continuous Delivery / Deployment (неперервна доставка/розгортання) — практика автоматизованого пакування перевіреного коду в релізні артефакти або його автоматичного розгортання на сервери.

40. Що таке GitHub Actions?

Вбудована платформа автоматизації робочих процесів (CI/CD) від GitHub, яка виконує сценарії на подійно керованих віртуальних машинах за конфігураційними YAML-файлами.

41. Що таке workflow?

Автоматизована процедура у GitHub Actions, описана у файлі .github/workflows/*.yml, що містить умови тригера запуску (on) та перелік завдань (jobs).

42. Що таке job?

Окремий блок завдань усередині workflow, що виконується на власному ізольованому runner-і послідовно крок за кроком (steps).

43. Що таке step?

Окремий неподільний крок усередині job, який або запускає shell-команду (run), або викликає готову дію з екосистеми GitHub Actions (uses).

44. Що таке runner?

Віртуальна машина або фізичний сервер (наприклад, ubuntu-latest), керований GitHub, який завантажує код репозиторію та виконує кроки job.

45. Коли повинен запускатися CI?

При кожній спробі внести зміни до репозиторію: події push у головні гілки (main, develop) та відкритті/оновленні pull request.

46. Для чого потрібний lint?

Для автоматизованого статичного аналізу коду без його запуску з метою виявлення синтаксичних дефектів, невикористаних імпортів, порушень стилю та потенційних логічних помилок.

47. Для чого використовується Ruff?

Надшвидкий лінтер та форматер для мови Python, написаний на Rust, який замінює цілий стек інструментів (Black, Flake8, isort, pyupgrade) і працює в десятки разів швидше.

48. Для чого потрібний mypy?

Інструмент статичної перевірки типів у Python, який аналізує type annotations (PEP 484) і знаходить помилки несумісності типів до моменту фактичного виконання коду.

49. Що таке type checking?

Процес верифікації типів даних аргументів функцій, повернених значень та атрибутів класів для запобігання типовим runtime-помилкам (TypeError, AttributeError).

50. Навіщо tests запускати в CI?

Щоб автоматично переконатися у відсутності регресій (помилок у вже працюючому коді) та заблокувати злиття некоректного коду у загальну гілку розробки.

51. Навіщо package build виконувати в CI?

Щоб перевірити, чи коректно сформовано конфігурацію pyproject.toml, чи присутні всі необхідні файли в маніфесті та чи може бути успішно зібрано валідний wheel/sdist.

52. Навіщо Docker build виконувати у CI?

Щоб гарантувати, що Dockerfile синтаксично правильний, залежності успішно встановлюються і сформований образ здатний зібратися на чистій цільовій машині.

53. Що таке GitHub Secret?

Зашифроване сховище змінних у налаштуваннях репозиторію GitHub, що дозволяє безпечно передавати токени та паролі у workflow без їх розкриття у відкритому тексті.

54. Як використовувати secret у workflow?

Використовуючи спеціальний вираз інтерполяції: ${{ secrets.ІМ'Я_СЕКРЕТУ }}, наприклад у секції env для конкретного кроку збирання.

55. Що таке documentation?

Комплекс технічних описів, інструкцій, специфікацій та прикладів, який пояснює призначення програми, архітектуру, методи налаштування, запуску та взаємодії з API.

56. Що повинен містити README?

Назву проєкту, детальний опис, перелік можливостей, системні вимоги, інструкцію зі встановлення, опис змінних оточення, команди запуску (локально та в Docker), опис API, запуск тестів, структуру та авторів.

57. Для чого потрібні docstrings?

Рядкові літерали на початку модулів, класів і функцій (PEP 257) для документування їх призначення, аргументів, типу результату та можливих винятків; використовуються функцією help() та Swagger.

58. Що таке OpenAPI?

Загальноприйнята машиночитана специфікація опису інтерфейсів REST API, що формалізує шляхи, методи, параметри запитів, тіла відповідей та моделі даних.

59. Що таке /docs у FastAPI?

Автоматично згенерований інтерактивний вебінтерфейс Swagger UI, що дозволяє досліджувати схему API, переглядати документацію моделей і тестувати виклики безпосередньо з браузера.

60. Що таке /openapi.json?

Стандартний системний маршрут FastAPI, який повертає повну схему поточного API у форматі JSON згідно зі специфікацією OpenAPI.

61. Що таке logging?

Систематизований запис подій, станів, інформаційних повідомлень і помилок під час роботи програми із фіксацією часу, рівня важливості та джерела події.

62. Чому logging краще за print?

Logging дозволяє керувати рівнями важливості (DEBUG, INFO, ERROR), фільтрувати повідомлення без зміни коду, перенаправляти потік у файли або зовнішні системи моніторингу та містить мітки часу.

63. Що таке production configuration?

Конфігурація застосунку, оптимізована для стабільності, максимальної швидкодії та безпеки: вимкнений debug/reload, суворе логування, безпечні секрети та персистентна БД.

64. Чим development відрізняється від production?

У development акцент зроблено на зручність дебагу (auto-reload, детальне трасування, in-memory бази), тоді як у production — на безпеку, стабільність, ізоляцію помилок та високу пропускну здатність.

65. Що таке Docker healthcheck?

Вбудована інструкція Dockerfile, яка періодично виконує команду всередині контейнера для верифікації того, що сервіс не завис і коректно обслуговує запити.

66. Для чого потрібний non-root user?

Для дотримання принципу найменших привілеїв: у разі зламу застосунку зловмисник не отримає прав root у контейнері та не зможе здійснити атаку на хост-систему (container escape).

67. Що таке multi-stage build?

Техніка оптимізації Dockerfile, за якої важкі білд-інструменти використовуються лише на проміжному етапі, а у фінальний компактний образ копіюються виключно готові артефакти.

68. Що таке coverage?

Метрика якості тестування, яка показує відсоток рядків/інструкцій вихідного коду програми, які були фактично виконані під час прогону тестового набору.

69. Навіщо встановлювати coverage threshold?

Щоб на рівні CI автоматично блокувати внесення нового або зміненого коду, який не має достатнього тестового покриття (наприклад, менше 80%), запобігаючи деградації надійності.

70. Що означає green CI pipeline?

Стан автоматизованого конвеєра перевірок, коли всі етапи (лінтинг, форматування, типізація, тести, збирання пакетів та контейнерів) завершилися зі статусом успіху (exit code 0).

14. Висновок

У ході виконання лабораторної роботи №10 навчальний проєкт «Система бронювання готелю (Hotel Reservation System)» (Варіант №8) було успішно переведено у повноцінний production-oriented стан відповідно до сучасних інженерних стандартів розробки.

Головні досягнення роботи:
1. Стандартизація та пакування: Проєкт оформлено як installable package відповідно до стандартів PEP 517, PEP 518 та PEP 621 з використанням pyproject.toml. За допомогою uv build та python -m build успішно зібрано бінарний дистрибутив wheel (hotel_booking-1.0.0-py3-none-any.whl) та вихідний архів sdist. Перевірено успішне встановлення wheel у чистому ізольованому середовищі;
2. Ізоляція конфігурації та безпека: Реалізовано методологію 12-Factor App за допомогою pydantic_settings.BaseSettings. Усі чутливі дані та параметри підключення до БД виносяться у змінні середовища. Створено публічний шаблон .env.example, а локальний файл .env надійно занесено до .gitignore. Проведено аудит безпеки, що підтвердив відсутність секретів у коді;
3. Високорівнева контейнеризація Docker: Створено багатоетапний Dockerfile (multi-stage build), який забезпечує запуск застосунку від безпечного непривілейованого користувача appuser та періодичний самоконтроль стану через HEALTHCHECK. Налаштовано персистентне збереження SQLite через змонтований том hotel_data у специфікації docker-compose.yml;
4. Покриття тестами та контроль якості: Розроблено тестовий набір із 36 тестів на базі pytest з ізольованою in-memory БД, що забезпечує 97% тестового покриття коду. Налаштовано суворий статичний аналіз коду через Ruff (linter + formatter) та строгу типізацію Муру (0 помилок у 11 модулях);
5. Повноцінний CI/CD Pipeline: У GitHub Actions налаштовано автоматичний конвеєр перевірки (.github/workflows/ci.yml) із тестовою матрицею Python 3.11 та 3.12, контролем порогу покриття (--cov-fail-under=80), автоматичним збиранням пакетів та формуванням перевіреного Docker-образу;
6. Завершення життєвого циклу курсу: Навчальний проєкт пройшов повну еволюцію від базової консольної програми (ЛР №1) до об'єктно-орієнтованої архітектури (ЛР №2–4), серіалізації (ЛР №5), тестування (ЛР №6), персистентності ORM (ЛР №7), асинхронного REST API (ЛР №8), профілювання та оптимізації (ЛР №9) і фінальної контейнеризованої production-системи (ЛР №10).

Проєкт повністю відповідає принципу: «Відтворити, автоматично перевірити, зібрати, контейнеризувати та запустити у стандартизованому середовищі» і готовий до фінального захисту.